++it Italian C++ Community

www.italiancpp.org

GIS a costo zero

Giorgio Bignotti

C++ Day 2026
October 17, Pavia

Estrarre un double




```
double x = *reinterpret_cast<const double*>(buffer + 5);
```

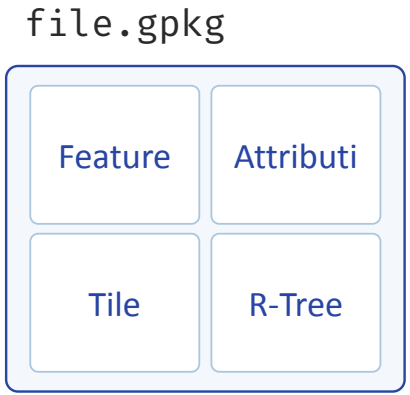
Estrarre un double



- Allineamento
- Endianness

GeoPackage

- Features: Point, LineString, Polygon, …
- Tiles: Mappe raster a piramide
- Attributi: tabelle senza geometria
- Estensioni: R-Tree, geometrie non lineari



GeoPackage

- Features: Point, LineString, Polygon, …
- Tiles: Mappe raster a piramide
- Attributi: tabelle senza geometria
- Estensioni: R-Tree, geometrie non lineari



GeoPackage

gpkg_contents

table_name	data_type	identifier
"miei_punti"	"features"	"miei_punti"
...	...	...

miei_punti

fid	geometry
1	BLOB
...	...

gpkg_geometry_columns

table_name	column_name	geometry_type_name
"miei_punti"	“geometry”	"POINT"
...	...	...

GeoPackage

```
GeoPackageBinaryHeader {
  byte[2]  magic = 0x4750
  byte     version
  byte     flags
  int32    srs_id
  double[] envelope
}

WKBGeometry{
  byte   byte_order
  uint32 geometry_type
  uint32 numPoints;
  Point
points[numPoints]
}

Point{
  double x
  double y
}
```



```
47 50
00
03
e6 10 00 00
83 f8 69 10 3f 0d 02 c0
f1 1d 58 be 03 cb f7 bf
76 05 15 af a0 e2 e5 3f
5a e4 5f 8b fc 6b f1 3f

01
02 00 00 00
02 00 00 00

83 f8 69 10 3f 0d 02 c0
5a e4 5f 8b fc 6b f1 3f
f1 1d 58 be 03 cb f7 bf
76 05 15 af a0 e2 e5 3f
```

GeoPackage

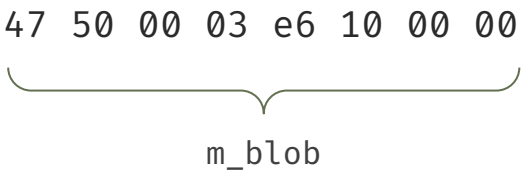
```
sqlite blob  ⟶  gpkg_header  ⟶  WKB  ⟶  GeometryView
```

- No copie
- No eccezioni
- Decodifica lazy

BinaryReader

```
class BinaryReader{
public:
    BinaryReader(std::span<const std::byte> blob, ByteOrder order);
    std::expected<std::span<const std::byte>, ParseError> read_bytes(std::size_t n);
    template<typename T>
    std::expected<T, ParseError> read();
private:
    std::span<const std::byte> m_blob;
    std::optional<ByteOrder> m_order;
}
```

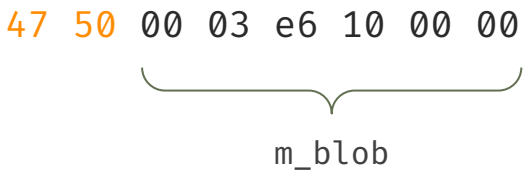
```
reader.read_bytes(2);    // ok
reader.read<int32_t>();  // ok
reader.read_bytes(4);    // error
```



BinaryReader

```
class BinaryReader{
public:
    BinaryReader(std::span<const std::byte> blob, ByteOrder order);
    std::expected<std::span<const std::byte>, ParseError> read_bytes(std::size_t n);
    template<typename T>
    std::expected<T, ParseError> read();
private:
    std::span<const std::byte> m_blob;
    std::optional<ByteOrder> m_order;
}
```

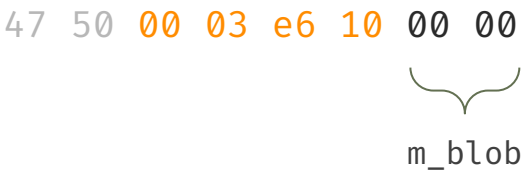
```
reader.read_bytes(2)      // ok
reader.read<int32_t>();   // ok
reader.read_bytes(4);     // error
```



BinaryReader

```
class BinaryReader{
public:
    BinaryReader(std::span<const std::byte> blob, ByteOrder order);
    std::expected<std::span<const std::byte>, ParseError> read_bytes(std::size_t n);
    template<typename T>
    std::expected<T, ParseError> read();
private:
    std::span<const std::byte> m_blob;
    std::optional<ByteOrder> m_order;
}
```

```
reader.read_bytes(2);    // ok
reader.read<int32_t>();  // ok
reader.read_bytes(4);    // error
```



BinaryReader

```
class BinaryReader{
public:
    BinaryReader(std::span<const std::byte> blob, ByteOrder order);
    std::expected<std::span<const std::byte>, ParseError> read_bytes(std::size_t n);
    template<typename T>
    std::expected<T, ParseError> read();
private:
    std::span<const std::byte> m_blob;
    std::optional<ByteOrder> m_order;
}
```

```
reader.read_bytes(2);     // ok
reader.read<int32_t>();   // ok
reader.read_bytes(4);     // error
```



BinaryReader

```
std::expected<std::span<const std::byte>, ParseError> read_bytes(std::size_t n){
    if (remaining() < n)
        return make_parse_error(ParseErrorCode::UnexpectedEof, "...");

    auto ret = m_blob.first(n);
    m_blob = m_blob.subspan(n);
    return ret;
}
```

```
std::expected<std::span<const std::byte>, ParseError>
```

BinaryReader

Estrarre un <T>

- Estrarre `sizeof(T)` byte
- Spostarli ad un indirizzo valido
- Cambiarne l’ordine

BinaryReader

Estrarre un <T>

- Estrarre `sizeof(T)` byte
- Spostarli ad un indirizzo valido
- Cambiarne l'ordine

```
read_bytes(sizeof(T))
```

BinaryReader

Estrarre un <T>

- Estrarre `sizeof(T)` byte
- Spostarli ad un indirizzo valido
- Cambiarne l'ordine

```
using U = std::conditional_t<sizeof(T) == 4,
          std::uint32_t, std::uint64_t>;

U raw;
std::memcpy(&raw, p, sizeof(raw));
```

BinaryReader

Estrarre un <T>

- Estrarre `sizeof(T)` byte
- Spostarli ad un indirizzo valido
- Cambiarne l'ordine

```
if(need_swap(order))
        raw = std::byteswap(raw);
```

BinaryReader

Estrarre un <T>

- Estrarre `sizeof(T)` byte
- Spostarli ad un indirizzo valido
- Cambiarne l'ordine

```
return std::bit_cast<T>(raw);
```

BinaryReader

```
template<WireScalar T>
T load(const std::byte* p, ByteOrder order)
{
    if constexpr (sizeof(T) == 1)
        return std::bit_cast<T>(*p);

    using U = std::conditional_t<sizeof(T) == 4, std::uint32_t, std::uint64_t>;
    U raw;
    std::memcpy(&raw, p, sizeof(raw));
    if (need_swap(order))
        raw = std::byteswap(raw);
    return std::bit_cast<T>(raw);
}
```

BinaryReader

```
template<typename T>
concept WireScalar =
    (std::integral<T> || std::floating_point<T> || std::same_as<T, std::byte>)
    && !std::same_as<T, bool>
    && (sizeof(T) == 1 || sizeof(T) == 4 || sizeof(T) == 8);
```

BinaryReader

```
template<WireScalar T>
std::expected<T, ParseError> read()
{
    if (sizeof(T) ≠ 1 && !m_order)
        return make_parse_error(ParseErrorCode::ByteOrderNotSet, "...");

    auto bytes = read_bytes(sizeof(T));
    if (!bytes)
        return std::unexpected(bytes.error());

    return load<T>(bytes→data(), m_order.value_or(ByteOrder::Little));
}
```

BinaryReader



x86-64 gcc 16.2, -std=c++23 -O2

```
double cast_read(const std::byte* p) {
    return *reinterpret_cast<const double*>(p);
}
```

```
"cast_read":
        movsd   xmm0, QWORD PTR [rdi]
        ret
```

```
double load_native(const std::byte* p) {
    return load<double>(p, ByteOrder::Little);
}
```

```
"load_native":
        movsd   xmm0, QWORD PTR [rdi]
        ret
```

BinaryReader



x86-64 gcc 16.2, -std=c++23 -O2

```
double cast_read(const std::byte* p) {
    return *reinterpret_cast<const double*>(p);
}
```

```
"cast_read":
        movsd   xmm0, QWORD PTR [rdi]
        ret
```

```
double load_swapped(const std::byte* p) {
    return load<double>(p, ByteOrder::Big);
}
```

```
"load_swapped":
        mov     rax, QWORD PTR [rdi]
        bswap   rax
        movq    xmm0, rax
        ret
```

BinaryReader



x86-64 gcc 16.2, -std=c++23 -O2

```cpp
double cast_read(const std::byte* p) {
    return *reinterpret_cast<const double*>(p);
}
```

```
"cast_read":
        movsd   xmm0, QWORD PTR [rdi]
        ret
```

```cpp
double load_runtime(const std::byte* p, ByteOrder o)
{
    return load<double>(p, o);
}
```

```
"load_runtime":
        mov     rdx, QWORD PTR [rdi]
        cmp     esi, 1
        mov     rax, rdx
        bswap   rax
        cmove   rdx, rax
        movq    xmm0, rdx
        ret
```

Propagazione dell’errore

```
std::expected<...> foo();

std::expected<...> bar(){

  auto var = foo();
  if(!var)
     return std::unexpected(var.error());

  // uso *var
}
```

```
std::expected<...> foo();

std::expected<...> bar(){

  ASSIGN_OR_PROPAGATE(var, foo());

  // uso var
}
```

Propagazione dell'errore

```
fn foo() → Result< ... >;

fn bar() → Result< ... >{

  let var = foo()?;

  // uso var
}
```

parse_gpkg_header

```
ASSIGN_OR_PROPAGATE(gp_bytes, reader.read_array<char, 2>());

if ((gp_bytes)[0] ≠ 'G' || (gp_bytes)[1] ≠ 'P')
    return make_parse_error( ... );
```

```
47 50
00
03
e6 10 00 00
83 f8 69 10 3f 0d 02 c0
f1 1d 58 be 03 cb f7 bf
76 05 15 af a0 e2 e5 3f
5a e4 5f 8b fc 6b f1 3f

01
02 00 00 00
02 00 00 00

83 f8 69 10 3f 0d 02 c0
5a e4 5f 8b fc 6b f1 3f
f1 1d 58 be 03 cb f7 bf
76 05 15 af a0 e2 e5 3f
```

parse_gpkg_header

```
GpkgHeader header;

ASSIGN_OR_PROPAGATE(version, reader.read<std::uint8_t>());
if (version ≠ 0)
    return make_parse_error( ... );

header.version = version;
```

```
47 50
00
03
e6 10 00 00
83 f8 69 10 3f 0d 02 c0
f1 1d 58 be 03 cb f7 bf
76 05 15 af a0 e2 e5 3f
5a e4 5f 8b fc 6b f1 3f

01
02 00 00 00
02 00 00 00

83 f8 69 10 3f 0d 02 c0
5a e4 5f 8b fc 6b f1 3f
f1 1d 58 be 03 cb f7 bf
76 05 15 af a0 e2 e5 3f
```

parse_gpkg_header

```
ASSIGN_OR_PROPAGATE(flags, reader.read<std::uint8_t>());

header.is_empty = ((flags_byte & 0b00010000) == 0b00010000);

if ((flags_byte & 0b00100000) ≠ 0)
    return make_parse_error( ... );

if ((flags_byte & 0b11000000) ≠ 0)
    return make_parse_error( ... );

if ((flags_byte & 0b00000001) == 0b00000001)
    reader.set_byte_order(ByteOrder::Little);
else
    reader.set_byte_order(ByteOrder::Big);
```

```
47 50
00
03
e6 10 00 00
83 f8 69 10 3f 0d 02 c0
f1 1d 58 be 03 cb f7 bf
76 05 15 af a0 e2 e5 3f
5a e4 5f 8b fc 6b f1 3f

01
02 00 00 00
02 00 00 00

83 f8 69 10 3f 0d 02 c0
5a e4 5f 8b fc 6b f1 3f
f1 1d 58 be 03 cb f7 bf
76 05 15 af a0 e2 e5 3f
```

parse_gpkg_header

```
ASSIGN_OR_PROPAGATE(srs_id, reader.read<std::int32_t>());
header.srs_id = srs_id;
```

```
47 50
00
03
e6 10 00 00
83 f8 69 10 3f 0d 02 c0
f1 1d 58 be 03 cb f7 bf
76 05 15 af a0 e2 e5 3f
5a e4 5f 8b fc 6b f1 3f

01
02 00 00 00
02 00 00 00

83 f8 69 10 3f 0d 02 c0
5a e4 5f 8b fc 6b f1 3f
f1 1d 58 be 03 cb f7 bf
76 05 15 af a0 e2 e5 3f
```

parse_gpkg_header

```
if (envelope_flag > 0) {
   ASSIGN_OR_PROPAGATE(base, reader.read_array<double, 4>());

    Envelope envelope = {.min_x = base[0],
                         .max_x = base[1],
                         .min_y = base[2],
                         .max_y = base[3]};

    if (envelope_flag == 2 || envelope_flag == 4) {
        //read Z coords
    }
    if (envelope_flag == 3 || envelope_flag == 4) {
        //read M coords
    }

    header.envelope = envelope;
}
```

```
47 50
00
03
e6 10 00 00
83 f8 69 10 3f 0d 02 c0
f1 1d 58 be 03 cb f7 bf
76 05 15 af a0 e2 e5 3f
5a e4 5f 8b fc 6b f1 3f

01
02 00 00 00
02 00 00 00

83 f8 69 10 3f 0d 02 c0
5a e4 5f 8b fc 6b f1 3f
f1 1d 58 be 03 cb f7 bf
76 05 15 af a0 e2 e5 3f
```

parse_wkb

```
ASSIGN_OR_PROPAGATE(order_byte, reader.read<std::byte>());

if(order_byte == std::byte{0})
    reader.set_byte_order(ByteOrder::Big);
else if(order_byte == std::byte{1})
    reader.set_byte_order(ByteOrder::Little);
else
    return make_parse_error( ... );
```

```
47 50
00
03
e6 10 00 00
83 f8 69 10 3f 0d 02 c0
f1 1d 58 be 03 cb f7 bf
76 05 15 af a0 e2 e5 3f
5a e4 5f 8b fc 6b f1 3f

01
02 00 00 00
02 00 00 00

83 f8 69 10 3f 0d 02 c0
5a e4 5f 8b fc 6b f1 3f
f1 1d 58 be 03 cb f7 bf
76 05 15 af a0 e2 e5 3f
```

parse_wkb

```
ASSIGN_OR_PROPAGATE(type_code, reader.read<std::uint32_t>());
ASSIGN_OR_PROPAGATE(dim, decode_dimensionality(type_code));

switch (type_code % 1000) {
case 2: { //LineString

    ASSIGN_OR_PROPAGATE(num_points, reader.read<std::uint32_t>());
    std::size_t total_bytes = num_points * coord_size(dim);

    ASSIGN_OR_PROPAGATE(bytes, reader.read_bytes(total_bytes));
    return LineStringView{CoordSequenceView(bytes, order, dim)};
}
```

```
47 50
00
03
e6 10 00 00
83 f8 69 10 3f 0d 02 c0
f1 1d 58 be 03 cb f7 bf
76 05 15 af a0 e2 e5 3f
5a e4 5f 8b fc 6b f1 3f

01
02 00 00 00
02 00 00 00

83 f8 69 10 3f 0d 02 c0
5a e4 5f 8b fc 6b f1 3f
f1 1d 58 be 03 cb f7 bf
76 05 15 af a0 e2 e5 3f
```

GeometryView

`std::string_view, std::span`

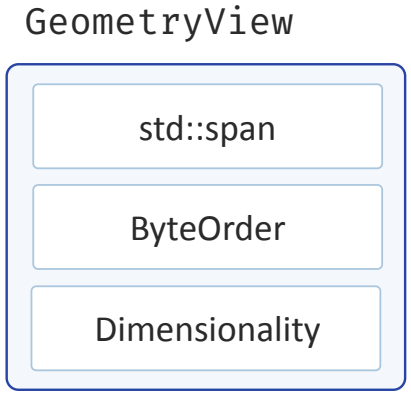
- Dove sono i dati
- Quanti sono

`GeometryView`

- Dove sono i dati
- Quanti sono
- Come leggerli

GeometryView

- Dimensione: 24 byte, indipendente dal numero di punti
- La decodifica avviene all’accesso



GeometryView

L’obiettivo:

```
GeometryView linestring = parse_wkb( ... );
for (auto p : linestring)
    std::println("[x: {}, y: {}]", p.x, p.y);
```

GeometryView

```
Point decode_point(const std::byte* p, ByteOrder order, Dimensionality dim)
{
    double x = load<double>(p, order);
    double y = load<double>(p + sizeof(double), order);

    Point pt{x, y, std::nullopt, std::nullopt};

    // z, m

    return pt;
}
```

GeometryView

```
class CoordSequenceView : public std::ranges::view_interface<CoordSequenceView>
{
public:
    CoordSequenceView( ... )

    std::size_t size() { return m_data.size() / coord_size(m_dim); }
    Point operator[](std::size_t i){
      return decode_point(m_data.data() + i * coord_size(m_dim), m_order, m_dim);
    }

private:
    std::span<const std::byte> m_data;
    ByteOrder m_order;
    Dimensionality m_dim;
};
```

GeometryView

```
using iterator = CoordSequenceIterator;
using const_iterator = CoordSequenceIterator;
```

```
Point operator*() const {
    return decode_point(m_ptr, m_order, m_dim);
}

CoordSequenceIterator& operator++(){
    m_ptr += step();
    return *this;
}
```

GeometryView

```
using iterator_concept = std::random_access_iterator_tag;
using iterator_category = std::input_iterator_tag;
using value_type = Point;
using reference = Point;
```

GeometryView

```
std::vector<double> xs = linestring | std::views::reverse
                                    | std::views::transform(get_x)
                                    | std::ranges::to<std::vector>();
```

Da dove arriva il blob?

`sqlite3_stmt` ⟶ `riga` ⟶ `blob` ⟶ `GeometryView`

- Il blob è memoria di SQLite
- Vale fino al prossimo `sqlite3_step`

Da dove arriva il blob?

- `for_each_feature()` fa `sqlite3_step()` → il blob è valido
- Chiama `callback(view)` → l'utente usa la view
- La callback ritorna → `for_each_feature()` fa `sqlite3_step()` → il blob precedente non è più valido

Garanzia o incentivo?

- La view nasce e muore dentro la callback
- La callback non può impedire la copia della view
- Se serve tenere i dati: `std::ranges::to`

Costo zero

Allocazioni per geometria valida	0
Byte copiati per LineString da N vertici	0 (fino a `std::ranges::to`)
Allocazioni su errore	1 (`std::string` errore)

Lezioni imparate

- `std::span` come cursore
- `memcpy + byteswap + bit_cast` per leggere senza UB
- `expected` per gestione errori esplicita
- View dice dove sono i dati

Grazie dell'attenzione

Contatti:

- giorgio.bigno@gmail.com
- github.com/GioBigno

